



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: _____12_____

Team No.: _____7_____

Project Title: _____Linux Concurrent Producer–Consumer System Using
Semaphores_____

Team Members

Roll Number	Student Name	Signature
2520090196	M.LAASYA SRI	Lasya sri
2520090197	M. RAJINI	Rajini
2520090213	K.RAMYA	Ramya

1. Abstract

The Producer–Consumer problem is a classic Operating System problem that demonstrates process synchronization and concurrent execution. In this project, a Linux-based Producer–Consumer system is developed using semaphores to coordinate the activities of producer and consumer processes. The producer generates data items and places them into a shared buffer, while the consumer retrieves and processes those items. Semaphores are used to control access to the shared buffer and prevent problems such as race conditions, buffer overflow, and buffer underflow. The project demonstrates important Operating System concepts including process synchronization, concurrency, critical sections, shared resources, mutual exclusion, and semaphore-based coordination. The system will be implemented using C programming on Linux/Ubuntu with POSIX/Linux APIs. The producer and consumer processes will execute concurrently, while synchronization mechanisms ensure safe and correct access to the shared buffer. The expected outcome is a functional Linux application that demonstrates how semaphores can be used to synchronize concurrent processes and efficiently manage shared resources

2. Problem Statement

In a multitasking operating system, multiple processes may access a shared resource at the same time. Without proper synchronization, simultaneous access can cause race conditions, data inconsistency, buffer overflow, and buffer underflow. The Producer–Consumer problem represents this situation, where a producer adds data to a shared buffer and a consumer removes data from it. This project aims to develop a Linux-based Producer–Consumer system using semaphores to synchronize the processes and ensure safe access to the shared buffer. The project demonstrates the importance of process synchronization and mutual exclusion in Operating Systems.

3. Objectives

(List the specific objectives of the project.)

1. To implement the Producer–Consumer problem in a Linux environment
2. To implement process synchronization using semaphores_
3. To manage a shared buffer between producer and consumer processes
4. To prevent race conditions, buffer overflow, and buffer underflow

4. Proposed Methodology

The project will be implemented using C programming on Linux/Ubuntu. A shared buffer will be created to store the data items produced by the producer and consumed by the consumer.

The producer process continuously generates data items and places them into the shared buffer. The consumer process retrieves the data items from the buffer and processes them. Semaphores will be used to control access to the shared buffer.

Three synchronization mechanisms can be used: a semaphore for empty buffer spaces, a semaphore for available filled items, and a mutex/binary semaphore for mutual exclusion. Before inserting an item, the producer checks for an available empty space and obtains exclusive access to the buffer. Similarly, before consuming an item, the consumer checks whether an item is available and obtains access to the shared buffer.

The system will display the producer and consumer activities through the Linux terminal and demonstrate synchronized concurrent execution.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Synchronization	Coordinates producer and consumer activities
Concurrency	Allows producer and consumer to execute concurrently
Semaphore	Controls access to shared resources.
Mutual Exclusion	Prevents simultaneous access to the critical section.
Critical Section	Protects the shared buffer from unsafe concurrent access.
Shared Buffer	Stores items produced before they are consumed.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090213	K.RAMYA	Implement producer process/thread and shared buffer management

2520090196	M.LAASYA SRI	Implement consumer process/thread and semaphore synchronization.
2520090197	M.RAJINI	Testing, debugging, documentation, PPT preparation, and project demonstration

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and execution environment.
C Programming	Development of the Producer–Consumer system.
GCC Compiler	Compiling the C program
POSIX Semaphores	Synchronization between concurrent execution units.
POSIX Threads	Creating producer and consumer threads, if implemented using threads.
Linux Terminal	Running and testing the application
GitHub	Managing and storing the project source code.

8. Expected Outcome

The project is expected to produce a functional **Linux-based Producer–Consumer system** that demonstrates concurrent execution and synchronization using semaphores. The producer will generate data items and place them into a shared buffer, while the consumer will retrieve and process the items. Semaphores will ensure that the producer does not add items when the buffer is full and the consumer does not remove items when the buffer is empty. The system will prevent race conditions and provide safe access to the shared resource. The project will provide practical understanding of **concurrency, process synchronization, critical sections, mutual exclusion, shared memory, and semaphores**.

9. GitHub Repository

GitHub Repository Link:

<https://github.com/laasya02678/klh-csit-2025-2029-07-producerconsumersemaphores/upload>

Repository Created: ____yes_____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: ____M.Raghupathi_____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____